

Possibly-New Findings

A step-by-step forensic audit of the “machine-verified proof of $P = NP$ ” in Lean 4 — what is genuinely new, how it was found, and how to recreate every check done by Daniele Coppeto

Repository audited	github.com/TiruArt/Pedigree-Polytopes-Lean4 (commit 3c9c90e, 2026-06-04)
Claim source	arXiv:2606.03194 — “The P versus NP Problem ... Pedigree Polytopes ...” (June 2026)
Toolchain	elan 4.2.4, Lean 4.30.0 (leanprover/lean4:v4.30.0), Mathlib from cache (8459 files)
Audit date / platform	2026-10-05, Windows 11, PowerShell, Python 3.14 (report tooling)
Scope	Findings A-D below, with discovery trail and verbatim recreation commands

Honesty statement (read this first)

This report uses a strict novelty legend. **NEW-CANDIDATE** means: we searched the public record (the repository's six GitHub issues, web search) on 2026-10-05 and did NOT find the finding documented — it is a candidate for new, not a proof of new. We cannot prove absence: other critiques may exist in places we did not read (partially JS-rendered issue comments, Reddit, Hacker News, X). **KNOWN** means prior art exists and is cited. None of this work advances the actual mathematics: P versus NP remains an unsolved problem, and nothing here changes that.

0. Executive summary

The repository and paper claim a machine-verified proof that $P = NP$. We audited the claim against the source code, the build system, the CI configuration, and the compiler itself. Four findings survived our novelty re-check. The headline of the whole audit: the central theorem is an assembly of undefined propositions and assumed axioms — but most of that was already known; what follows is what might not be.

ID	Finding (one line)	Status	Confidence it is new
A	The polytope A_n that the optimization chain runs on is literally the one-point type <code>Unit</code> (author's own “placeholder” comment in source)	NEW-CANDIDATE	Medium
B	Fresh clone cannot build the full library (file never committed); default build compiles 0 targets, so CI is green without compiling anything	NEW-CANDIDATE	Medium-high
C	The paper advertises “theorem <code>p_equals_np</code> : <code>P_class = NP_class</code> ”, but the repo's theorem proves an opaque axiom and <code>P_class/NP_class</code> exist nowhere in the code	NEW-CANDIDATE	Medium
D	Compiler-generated <code>#print</code> axioms receipts (21 dependencies) for the current HEAD	KNOWN in substance; receipt unpublished	Low

Section 6 lists what we explicitly retract as NOT new (found before us), Section 7 is the full from-scratch reproduction guide, Section 8 the limitations.

1. How we got there — overall method

The audit followed one rule: **never trust the paper's description of the code — read the code, then let the compiler speak**. The trail, in order:

1. Read the arXiv paper (2606.03194) end to end and extracted its checkable claims (“36 Lean 4 files, 2968/2968 build targets clean”, “zero unresolved sorrys in the main proof chain”, the displayed theorem statement).

2. Cloned the linked GitHub repository (194 .lean files at commit 3c9c90e) and inspected the sources statically with ripgrep searches (definitions of P, NP, polynomial time, Turing machines, axioms, sorrys, placeholders).
3. Installed the Lean toolchain (elan) exactly as a third party would, fetched the Mathlib olean cache, and built the theorem's module chain to reproduce the paper's numbers.
4. Wrote a 10-line Audit.lean and ran #print axioms — the Lean kernel's own list of everything the theorem rests on (Finding D).
5. While debugging the build (the receipts required a working build), walked into Findings A and B; while diffing paper claims against code, found Finding C.
6. Searched the public record (GitHub issues, web search) to separate what we had independently rediscovered from what was genuinely undocumented — this report's NEW-CANDIDATE labels are the result of that pass, and Section 6 records our retraction of earlier, premature “new discovery” labels.

2. Finding A — the polytope A_n is the one-point type Unit

Status: NEW-CANDIDATE (medium confidence)

What it is

The paper's argument optimizes over the polytope $\text{conv}(A_n)$ (A_n = the projected pedigree polytope in Euclidean space). In the Lean source, the type that parameterizes this entire chain is a one-element placeholder:

```
-- MembershipProject/Core/N_PEqualsNP.lean, lines 145-146
/-- The projected polytope  $A_n$  (placeholder type). -/
def An (n : ℕ) : Type := Unit -- placeholder:  $\mathbb{R}^{\{\alpha_n\}}$  vectors
```

Unit is Lean's empty-tuple type: it has exactly one element. Every type-level “polytope” in the chain — including the arguments to the axioms `mi_objective_solves_stsp_ax`, `convAn_full_dimensional_ax`, `convAn_has_interior_point_ax` and `convAn_rationality_guaranteed_ax` — is this one-point type, for every n . The predicates applied to it (`PolynomialOptimisation`, `FullDimensional`, ...) are themselves undefined opaque axioms, so nothing is ever computed on it: the symbol A_n flows through the proof as a name only.

Discovery trail (step by step)

1. Paper reading: the abstract claims the optimization step runs over $\text{conv}(P_n) / \text{conv}(A_n)$ and that all main-chain proofs are machine-verified with zero sorrys.
2. Clone the repo, open the file that ends the chain (`Core/N_PEqualsNP.lean`, which contains theorem `p_equals_np`), read it top-down rather than grepping — line 145 stops you, because the docstring says “placeholder type” in the author's own words.
3. Line 146 confirms the value: `Type := Unit`, with an inline comment naming what it should have been ($\mathbb{R}^{\{\alpha_n\}}$ vectors).
4. Trace consumers of `An`: `grep` for “`An n`” shows it appears only as an argument inside axiom declarations and their applications — never in a definition that computes anything.
5. Compiler cross-check: add “`#check @MembershipProject.Core.An`” to `Audit.lean`; Lean prints `An : ℕ → Type` (the source line shows the value `Unit`).
6. Novelty pass: search the six GitHub issues and the web for “placeholder”, “Unit”, “ A_n ”. Issue #1 discusses placeholder PROPOSITIONS (`def P_equals_NP := True`) from an earlier revision, not this placeholder TYPE. No hit found → NEW-CANDIDATE, medium (issue comment threads are partly JS-rendered, so we cannot claim certainty).

How to recreate (2 minutes, no Lean needed)

```
git clone https://github.com/TiruArt/Pedigree-Polytopes-Lean4.git
cd Pedigree-Polytopes-Lean4
Select-String -Path "MembershipProject\Core\N_PEqualsNP.lean" -Pattern "placeholder"
```

Expected: two hits, lines 145 and 146, showing “placeholder type” and “def An (n : $\mathbb{N}$) : Type := Unit”.
Optional compiler confirmation: see Appendix, Steps 1-7 (the #check line in Audit.lean).

3. Finding B — build and CI forensics

Status: NEW-CANDIDATE (medium-high confidence)

What it is

Three interlocking facts about building the repository from a fresh clone:

- **A live module was never committed.** Two source files import MembershipProject.Core.F5Network, but that file does not exist in the repository — only Backup/F5Network.lean and Backup/F5Network1.lean do. git ls-files confirms it was never tracked at any commit.
- **The full-library build therefore fails.** “lake build MembershipProject” dies in job computation with: no such file or directory — MembershipProject\Core\F5Network.lean.
- **The default build compiles nothing.** Plain “lake build” prints “Build completed successfully (0 jobs)” and exits 0 — in our local runs, before and after the dependency setup. The repository's CI (.github/workflows/lean_action_ci.yml) runs leanprover/lean-action@v1 with defaults, i.e. this same default build command — so a green CI badge is not evidence that the library compiles.

Counter-evidence we also verified (being fair to the authors)

The paper's specific number — 2968/2968 build targets clean — DOES reproduce, but only when you build the main theorem's target explicitly: “lake build MembershipProject.Core.N_PEqualsNP” compiled 2968 of 2968 targets with zero errors, and the separate necessity module built 2960/2960. So the proof chain itself is buildable; it is the full library, the default build, and the CI signal that are broken. We did not execute GitHub Actions itself — the CI conclusion is based on reading the workflow file plus local runs of the same default command.

Discovery trail (step by step)

1. Goal: run #print axioms. First attempt, “lake env lean Audit.lean”, fails with “unknown module prefix MembershipProject” — nothing had been built yet.
2. Ran plain “lake build” (what CI runs): exit code 0, but “0 jobs” — and still no oleans on disk. A build that claims success while compiling nothing is a red flag.
3. Ran the explicit library target “lake build MembershipProject”: hard error on MembershipProject\Core\F5Network.lean — a file the job computation needs.
4. Grep for F5Network: only Backup/ copies and import statements; “git ls-files | Select-String F5Network” lists ONLY the two Backup files → never committed.
5. Who imports the ghost module: Tests/TestPythonIntegration.lean (line 1) and Algorithms/PythonMaxflow.lean (line 3), plus two extensionless copies.
6. Read .github/workflows/lean_action_ci.yml: leanprover/lean-action@v1 with no options → the default build command.
7. Workaround for the audit: build the theorem's own module target → 2968/2968 success (paper's number reproduced — recorded above as counter-evidence).
8. Novelty pass: searched F5Network / “0 jobs” / CI critiques — zero hits → NEW-CANDIDATE, medium-high (most checkable finding in this report).

How to recreate (10 minutes)

```
cd Pedigree-Polytopes-Lean4

# 1. The ghost file: only Backup copies are tracked
git ls-files | Select-String F5Network
#   -> Backup/F5Network.lean , Backup/F5Network1.lean   (Core/ version absent)

# 2. Who imports the missing module
Get-ChildItem -Recurse -Filter *.lean | Select-String -Pattern "import MembershipProject.Core.F5Network"
#   -> Tests\TestPythonIntegration.lean:1 , Algorithms\PythonMaxflow.lean:3

# 3. Default build: success with zero work
lake build
#   -> 'Build completed successfully (0 jobs).'   (exit code 0)

# 4. Full library build: fails on the missing file
lake build MembershipProject
#   -> error: no such file or directory ... MembershipProject\Core\F5Network.lean

# 5. Main theorem target: reproduces the paper's number
lake build MembershipProject.Core.N_PEqualsNP
#   -> [2968/2968] ... Build completed successfully (2968 jobs).
```

4. Finding C — the theorem statement in the paper does not exist in the repo

Status: NEW-CANDIDATE (medium confidence)

What it is

The arXiv paper (2606.03194) displays its headline result twice — in the abstract block and in the body — as:

```
theorem p_equals_np
: P_class = NP_class
```

The repository contains no identifier `P_class` or `NP_class` anywhere (zero matches across all 194 .lean files). The actual theorem is a different statement with a different conclusion:

```
-- MembershipProject/Core/N_PEqualsNP.lean
line 107: axiom P_equals_NP : Prop          -- an opaque, undefined constant
...
line 364: theorem p_equals_np (n : ℕ) (hn : 5 ≤ n) : P_equals_NP := by
```

So the statement the paper advertises (equality of two complexity classes) is not what was formalized; what was formalized is “derive the opaque constant `P_equals_NP` from other opaque constants”. Tellingly, the author's own GitHub reply explains that a real proof “would require defining `P_class` and `NP_class` over a formal model of computation” — the names exist in the paper, were never defined in code, and the paper nonetheless prints them inside the theorem statement.

Discovery trail (step by step)

1. Method chosen up front: build a claim-by-claim matrix of paper vs. repo (every displayed theorem, every number).
2. Saved the full text of arXiv:2606.03194 to a local file for grepping.
3. `grep` the paper text for “`P_class`” → exactly 2 hits: the abstract display block and the body display block, both as “`theorem p_equals_np : P_class = NP_class`”.
4. `grep` every .lean file in the repo for “`P_class|NP_class`” → zero hits.
5. Read the real theorem: line 364 concludes `P_equals_NP`, which line 107 declares as an axiom (undefined `Prop`).
6. Cross-reference with issue #1: commenters quote the REPO statement (the opaque-axiom version) and the author discusses `P_class`/`NP_class` as future work — but nobody flags that the paper prints a theorem statement that the repo does not contain → NEW-CANDIDATE, medium (again: absence of evidence, not evidence of absence).

How to recreate (5 minutes)

```
cd Pedigree-Polytopes-Lean4

# 1. The repo side: P_class / NP_class do not exist
Get-ChildItem -Recurse -Filter *.lean | Select-String -Pattern "P_class|NP_class"
# -> no output

# 2. What the theorem really says
Select-String -Path "MembershipProject\Core\N_PEqualsNP.lean" -Pattern "axiom P_equals_NP|theorem p_equals_np"
# -> line 107: axiom P_equals_NP : Prop
# -> line 364: theorem p_equals_np (n : ℕ) (hn : 5 ≤ n) : P_equals_NP := by

# 3. The paper side: open https://arxiv.org/abs/2606.03194 , download the source or HTML,
# search the text for 'P_class' -> appears inside the displayed theorem twice
```

5. Finding D — the compiler-generated receipts (#print axioms)

Status: KNOWN in substance (GitHub issue #1 concluded the same thing manually); this machine-generated list appears unpublished — LOW novelty

Discovery trail (step by step)

1. Builds were working (Appendix Steps 1-7), so we wrote a 10-line file asking Lean itself what the headline theorem depends on:

```
import MembershipProject.Core.N_PEqualsNP
import MembershipProject.Core.N_MembershipCharacterisation

-- The receipts: what does the "machine-verified P = NP" actually depend on?
#print axioms MembershipProject.Core.p_equals_np
#print axioms MembershipProject.Core.necessity_of_mcf
#print axioms MembershipProject.Core.mi_objective_solves_stsp

-- And what is the polytope An the optimization runs on?
#check @MembershipProject.Core.An
```

Ran "lake env lean Audit.lean". Verbatim output:

```
'MembershipProject.Core.p_equals_np' depends on axioms: [proptext,
Quot.sound,
MembershipProject.Core.FullDimensional,
MembershipProject.Core.HasInteriorPoint,
MembershipProject.Core.P_equals_NP,
MembershipProject.Core.PolynomialOptimisation,
MembershipProject.Core.PolynomialSeparationOracle,
MembershipProject.Core.QuickProtocol,
MembershipProject.Core.RationalityGuaranteed,
MembershipProject.Core.SAT_in_P,
MembershipProject.Core.STSP_in_P,
MembershipProject.Core.convAn_full_dimensional_ax,
MembershipProject.Core.convAn_has_interior_point_ax,
MembershipProject.Core.convAn_rationality_guaranteed_ax,
MembershipProject.Core.cook_np_completeness,
MembershipProject.Core.gls_optimisation,
MembershipProject.Core.karp_stsp_np_complete,
MembershipProject.Core.maurras_separation,
MembershipProject.Core.membership_An_of_Pn,
MembershipProject.Core.mi_objective_solves_stsp_ax,
MembershipProject.Core.tardos_strongly_polynomial]
'MembershipProject.Core.necessity_of_mcf' depends on axioms: [proptext,
Classical.choice,
Quot.sound,
MembershipProject.Core.necessity_of_mcf]
'MembershipProject.Core.mi_objective_solves_stsp' depends on axioms: [MembershipProject.Core.PolynomialOptimisation,
MembershipProject.Core.STSP_in_P,
MembershipProject.Core.mi_objective_solves_stsp_ax]
MembershipProject.Core.An : ℕ → Type
```

How to read it

Category	Count	Items
Standard logical axioms (harmless)	2	propext, Quot.sound (+ Classical.choice in the second query)
Undefined atoms — props with no definition	9	P_equals_NP (the thing being “proven” itself!), STSP_in_P, SAT_in_P, QuickProtocol, PolynomialOptimisation, PolynomialSeparationOracle, FullDimensional, RationalityGuaranteed, HasInteriorPoint
The project's novel math, assumed not proved	5	mi_objective_solves_stsp_ax (the STSP reduction), membership_An_of_Pn, convAn_full_dimensional_ax, convAn_has_interior_point_ax, convAn_rationality_guaranteed_ax
Legit published external results (citations)	5	cook_np_completeness, karp_stsp_np_complete, tardos_strongly_polynomial, gls_optimisation, maurras_separation

- The most striking line: **P_equals_NP appears in its own proof's axiom list** — because the proposition being proved was declared by “axiom P_equals_NP : Prop”, the kernel counts it as an assumption of the result.
- necessity_of_mcf prints **itself** as its dependency — Lean's way of saying “I am an axiom; I was assumed”. (Consistent with the paper admitting the necessity direction is future formalisation.)
- The necessity module is not even imported by the final theorem's chain — it had to be built separately.

6. What is NOT new — prior art and our retraction

Our first pass labelled several things “new discovery”. Checking the public record showed we had independently rediscovered an already-published critique. Retracting correctly:

Our earlier claim	Found before us by	When
“The theorem does not prove $P = NP$ ” (opaque atoms, no formalization of P/NP)	GitHub issue #1 “Main theorem currently proves True, not $P = NP$ ” (user log2cn): caught def P_equals_NP : Prop := True — the theorem literally proved True. The author then converted the placeholders to opaque axioms; a commenter wrote “So it proves an axiom is true now.”	2026-06-03 (before this audit)
“ P/NP are never defined over a model of computation”	The repo author's own reply in issue #1: a fully formal proof “would require defining P_class and NP_class over a formal model of computation” — conceded as beyond scope	June 2026
“The chain is axioms, not proofs”	Public in the repository README (pipeline steps labelled “Axiom”)	always public
“The necessity direction is unproved”	The paper itself admits it (future formalisation)	June 2026

“Thank you for this precise observation. You are correct: Theorem p_equals_np now derives P_equals_NP from a chain of axioms — and since P_equals_NP is itself an opaque axiom, the theorem establishes that this axiom is reachable via the logical chain, not that $P = NP$ in a formally defined complexity-theoretic sense.”

— repository author, GitHub issue #1 (excerpt as surfaced in web search, retrieved 2026-10-05)

Reading note for Finding D: the substance (axiom chain, undefined atoms) was concluded by humans in June 2026 without the tool. What we add is only the kernel-generated receipt at the current commit — hence LOW novelty, honestly labelled.

7. Appendix — full reproduction from a clean machine

Every command below is PowerShell on Windows; on Linux/macOS replace Select-String with grep and the path separators. Total time: roughly 30-60 minutes, dominated by the Mathlib cache download (8459 files, several GB).

Step 1 — install elan (Lean version manager)

```
Invoke-WebRequest https://raw.githubusercontent.com/leanprover/elan/master/elan-init.ps1 -OutFile elan-init.ps1
.\elan-init.ps1 -y --default-toolchain none
$env:Path = "$env:USERPROFILE\.elan\bin;$env:Path"
elan --version # we used 4.2.4
# Note: we originally fetched the elan release zip manually because the bare .exe URL 404'd;
# the script above is the standard route and does the same thing.
```

Step 2 — clone the audited repository

```
git clone https://github.com/TiruArt/Pedigree-Polytopes-Lean4.git
cd Pedigree-Polytopes-Lean4
git log -1 --format="%h %ad %s" --date=short
# -> 3c9c90e 2026-06-04 Simplify Lean Action CI workflow (state audited in this report)
cat lean-toolchain
# -> leanprover/lean4:v4.30.0 (lake will auto-install it on first run)
```

Step 3 — fetch the Mathlib olean cache

```
lake exe cache get
# -> downloads ~8459 prebuilt Mathlib files (multi-GB). Expect 10-30 min on a home line.
# Without this step, building Mathlib from source takes many hours.
```

Step 4 — build the theorem's module chain

```
lake build MembershipProject.Core.N_PEqualsNP
# -> [2968/2968] Build completed successfully (2968 jobs). <- the paper's number

lake build MembershipProject.Core.N_MembershipCharacterisation
# -> [2960/2960] success (needed for necessity_of_mcf; NOT in the final theorem's import chain)
```

Step 5 — write Audit.lean (contents shown in Finding D)

Create a file named Audit.lean at the repository root with the ten lines from Finding D (two imports, three #print axioms, one #check).

Step 6 — run the audit

```
lake env lean Audit.lean
# -> the verbatim receipts printed in Finding D, exit code 0
```

Step 7 — reproduce Findings A, B, C

Each finding's section contains its own “How to recreate” block with exact commands and expected outputs. Cheat sheet:

Finding	One-line check	Expected
A	Select-String -Path "MembershipProject\Core\N_PEqualsNP.lean" -Pattern placeholder	lines 145-146: “placeholder type”, “def An (n : ℕ) : Type := Unit”
B1	git ls-files Select-String F5Network	only Backup/ paths — no Core file
B2	lake build then lake build MembershipProject	0 jobs + exit 0 then hard error on missing F5Network.lean
C	Get-ChildItem -Recurse -Filter *.lean Select-String -Pattern "P_class NP_class"	no output (vs. paper's displayed “: P_class = NP_class”)

Findin g	One-line check	Expected
D	lake env lean Audit.lean	21-item axiom list; P_equals_NP inside it

Troubleshooting we hit

- “unknown module prefix 'MembershipProject'” — the project oleans are not built yet; run Steps 3-4 first (plain “lake build” will not do it; see Finding B).
- “object file ... N_MembershipCharacterisation.olean does not exist” — build that module explicitly (Step 4); it is not a dependency of the final theorem.
- elan not found — re-open the shell or re-run the \$env:Path line of Step 1.
- Disk space: Mathlib cache plus build artifacts take several GB; keep that in mind before starting.

8. Limitations of this report

- **We cannot prove novelty.** NEW-CANDIDATE means “not found in our searches on 2026-10-05”: six GitHub issues (titles and bodies; some comment threads are JavaScript-rendered and could not be fully read), plus general web search. We did not exhaust Reddit, Hacker News, X/Twitter, or the papers citing arXiv:2606.03194.
- **CI behaviour is inferred, not executed.** We read the workflow file and reproduced its default build command locally (0 jobs, exit 0); we did not run GitHub Actions itself, so the badge status on the repository was not re-tested.
- **The 0-jobs mechanism is observed, not root-caused.** We report the exact behaviour and the commands; why Lake resolves the default target to an empty set in this package was not fully investigated.
- **Static audit only for the sound parts.** The 2968 build targets that do compile are real Lean code checked by the kernel; this report does not challenge their internal correctness — only what the overall theorem claims to establish.
- **P versus NP is still open.** Nothing in this audit is evidence that P equals NP or that P differs from NP. It is evidence that one particular claimed machine-verified proof does not constitute a proof of either.

Bottom line

Findings A-C are our best candidates for genuinely undocumented observations about a public claim (re-check them before publishing — they are cheap to verify and expensive to get wrong); Finding D is corroboration with compiler receipts; everything else we first called “new” was found before us and is credited in Section 6.